

Die Vitalzeichen einer Plattform: Wie man mit den Four Golden Signals den Gesundheitszustand von frag.jetzt überwacht

Exposé zur Bachelorarbeit im Studiengang Informatik

Max Mustermann

27. Januar 2026

Zusammenfassung

Moderne Webplattformen wirken nach außen oft stabil und reaktionsschnell, doch ihr innerer Zustand bleibt im laufenden Betrieb häufig verborgen. Dieser Artikel zeigt, wie Observability dabei hilft, verteilte Systeme wie die Open-Source-Plattform *frag.jetzt* als Ganzes zu verstehen und ihren „Gesundheitszustand“ sichtbar zu machen. Anhand der Four Golden Signals wird diskutiert, welche technischen Anzeichen tatsächlich aussagekräftig sind, warum isoliertes Monitoring nicht ausreicht und wie Metriken, Logs, Traces und intelligentes Alerting zusammenspielen müssen, um handlungsrelevante Einblicke zu liefern. Im Fokus steht dabei weniger die Theorie einzelner Werkzeuge als vielmehr die Frage, wie aus Rohdaten Entscheidungen werden, die Betrieb, Entwicklung und Produktverantwortung gleichermaßen unterstützen.

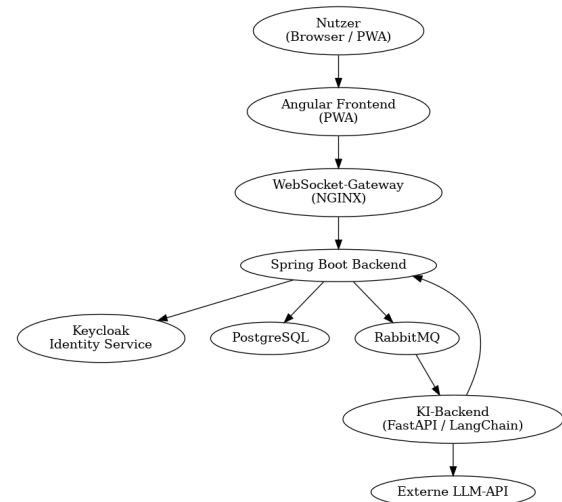


Abbildung 1: Das Nervensystem des Patienten: Der vereinfachte Anfragepfad bei frag.jetzt [1].

1 Einleitung

Stellen Sie sich folgende Szene vor: Ein vollbesetzter Hörsaal, 500 Studierende starren auf die Leinwand. Der Dozent hat gerade eine interaktive Fragerunde gestartet. 500 Daumen tippen auf Smartphones. Absenden.

Und dann: **Stille.**

Auf der Leinwand dreht sich der Ladebalken. Sekunden vergehen, die sich wie Minuten anfühlen. Ist das WLAN der Uni zusammengebrochen? Hat sich der Datenbank-Pool erschöpft? Oder wartet unser Backend gerade vergeblich auf eine Antwort der externen KI-Schnittstelle?

In der „guten alten Zeit“ monolithischer Software hätten wir in *eine* Log-Datei geschaut und die Antwort gefunden. Aber *frag.jetzt* ist heute ein modernes, cloud-natives Nervensystem: Ein Angular-Frontend spricht mit einem NGINX-Gateway, das Anfragen an ein Spring-Boot-Backend weiterreicht, welches wiederum Keycloak zur Identitätsprüfung anruft, Nachrichten in RabbitMQ schiebt und schließlich komplexe Berechnungen an ein Python-basiertes KI-Backend auslagert [1].

Wenn hier etwas hakt, ist das System selten komplett „tot“ – es ist „krank“. Es leidet an unsichtbaren Entzündungen: Ein einzelner Service antwortet 200 Millisekunden zu spät und löst eine Kettenreaktion aus, die sich bis zum Nutzer als völliger Stillstand aufstaut [2].

Das Problem: Herkömmliches Monitoring sagt uns oft nur: „Der Server ist an.“ Es sagt uns nicht, *warum* sich die Anwendung für den Nutzer zäh anfühlt. Hier wechseln wir die Strategie – von der einfachen Überwachung zur **Observability**. Wir wollen nicht mehr nur wissen, ob das Herz noch schlägt, sondern wie gesund der Patient wirklich ist. Dafür nutzen wir die *Four Golden Signals* als unsere medizinischen Vitalwerte [3].

Dieser Artikel zeigt, wie wir *frag.jetzt* in die Diagnose-Röhre geschoben haben, um aus stummen Daten sprechende Antworten zu machen – und warum „Ich glaube, es hängt“ keine gültige Fehlerbeschreibung mehr ist.

2 Vier Golden Signals: Der Gesundheits-Check statt Symptom-Raten

Wenn ein Patient in die Notaufnahme kommt, misst der Arzt nicht „alles“. Er schaut auf die Vitalwerte. In der Welt der Google Site Reliability Engineers (SRE) haben sich dafür vier universelle Metriken etabliert, die sogenannten *Four Golden Signals* [3]. Für *frag.jetzt* bedeuten sie den Unterschied zwischen „Ich glaube, es läuft“ und „Ich weiß, dass es läuft“.

Hier ist, was diese Signale für uns konkret bedeuten:

2.1 Latenz: Die Zeit zwischen Klick und Erlösung

Es ist egal, dass die Datenbank die Anfrage in 5 Millisekunden verarbeitet hat, wenn der Nutzer im Hörsaal über schlechtes WLAN erst nach 3 Sekunden eine Antwort sieht. Latenz ist die Zeit, die es braucht, eine Anfrage zu bedienen.

Der Trugschluss: Der Durchschnitt lügt. Wenn 90 % der Anfragen schnell sind, aber die langsamsten 10 % (die Dozenten-Anfragen?) hängen, haben wir ein Problem. Wir messen daher nicht nur den Mittelwert, sondern die *Tail Latency* – also wie lange die langsamsten Anfragen dauern. Denn genau dort entsteht der Frust [4].

2.2 Traffic: Der Herzschlag des Hörsaals

Wie viel Last trifft unser System gerade? Bei *frag.jetzt* ist das kein stetiges Rauschen, sondern ein Tsunami.

Das Szenario: Die Vorlesung beginnt. 500 Studierende öffnen gleichzeitig die App. Der Traffic schießt von 0 auf 500 Anfragen pro Sekunde. Ein plötzlicher Abfall auf Null ist hier übrigens genauso verdächtig wie eine Überlastung – oft bedeutet er nicht „Feierabend“, sondern „Netzwerkfehler“.

2.3 Fehler: Wenn das Schweigen laut wird

Klassische Fehler sind einfach: Ein HTTP-Code 500 (Internal Server Error) ist ein Schrei nach Hilfe.

Die Tücke: Viel gefährlicher sind die „leisen“ Fehler. Wenn unser Backend zwar „Code 200 OK“ antwortet, aber der Inhalt leer ist oder falsche Daten liefert, merkt das Monitoring nichts – der Nutzer aber alles. Observability bedeutet hier, tief genug in die Antwort zu schauen, um auch funktionale Pannen zu erkennen.

2.4 Sättigung: Der Stau vor der Kasse

Das ist der vielleicht wichtigste Indikator für die Zukunft. Sättigung misst, wie „voll“ unsere Ressourcen sind [3].

Die Analogie: Stellen Sie sich die Warteschlange in der Mensa vor. Solange die Schlange sich bewegt,

ist die Latenz okay. Aber wenn die Mensa zu 99 % gefüllt ist, reicht eine einzige weitere Person, und alles kommt zum Stillstand. Sättigung (CPU-Last, Speicher, Datenbank-Pools) warnt uns, *bevor* das System langsam wird. Sie ist unser Frühwarnsystem.

Fazit der Diagnose: Ein einzelner Wert ist nutzlos. Hohe Latenz kann an hoher Sättigung liegen – oder an einem Netzwerkfehler. Erst wenn wir diese vier Signale übereinanderlegen (Korrelation), wird aus Daten echtes Wissen.

3 Werkzeug-Tetris: Die Suche nach dem perfekten Stack

Wer heute „Observability Tools“ in eine Suchmaschine eingibt, wird von einer Lawine begraben. Es gibt hunderte Dienste, vom teuren Enterprise-SaaS bis zum selbstgehackten Skript. Für ein Open-Source-Projekt wie *frag.jetzt* war klar: Wir können uns keine Lizenzkosten leisten, aber wir können uns auch keine blinden Flecken leisten [5].

Wir standen vor der klassischen Frage: Was brauchen wir wirklich? Einen Detektiv, einen Wächter oder ein Navigationssystem? Die Antwort lautet: Alles drei – aber in der richtigen Dosierung.

3.1 1. Der Wächter: Prometheus für den Puls

Unsere erste Wahl fiel auf **Prometheus**. Warum? Weil Logs teuer sind. Jede Zeile Text zu speichern, kostet Speicherplatz und Rechenleistung. Metriken hingegen sind billig. Prometheus speichert keine Romane, sondern Zahlenreihen: „CPU bei 80 %“, „50 Anfragen pro Sekunde“ [6]. Das ist unser EKG. Es sagt uns sofort, *dass* der Patient Stress hat – aber nicht unbedingt warum.

3.2 2. Der Detektiv: Logs (aber bitte sparsam)

Wenn Prometheus Alarm schlägt, brauchen wir Details. Hier kommt klassisch der **ELK-Stack** (Elasticsearch, Logstash, Kibana) ins Spiel. Aber Vorsicht: In verteilten Systemen explodieren Logs förmlich. Wir haben gelernt: „Log everything“ ist keine Strategie, sondern ein Weg, die Festplatte zu verstopfen. Wir nutzen Logs daher gezielt für Fehlermeldungen und den Applikationsstart, nicht für jeden Herzschlag [7].

3.3 3. Das Navi: Distributed Tracing mit Jaeger

Hier wurde es für *frag.jetzt* kritisch. Metriken und Logs sind isoliert. Sie sagen: „Die Datenbank ist langsam“ und „Das Frontend wartet“. Aber sie sagen nicht: „Das Frontend wartet, *weil* die Datenbank langsam ist.“

In einer Microservice-Architektur wie unserer verlieren wir ohne **Tracing** den Faden. Wir haben uns

für **Jaeger** entschieden, um den Weg einer einzelnen Anfrage quer durch NGINX, Spring Boot und das KI-Backend nachzuzeichnen. Erst Tracing macht aus isolierten Komponenten eine zusammenhängende Geschichte [8].

Die Geheimwaffe: OpenTelemetry

Das größte Risiko bei der Tool-Auswahl ist der „Vendor Lock-in“. Was, wenn wir morgen von Jaeger zu Tempo wechseln wollen? Oder von Prometheus zu einer Cloud-Lösung? Unsere Strategie setzt auf **OpenTelemetry (OTel)** als universellen Adapter. Statt unseren Code mit Prometheus-spezifischen Bibliotheken zu „verschmutzen“, instrumentieren wir alles neutral mit OTel. OTel sammelt die Daten ein und verteilt sie an die Tools unserer Wahl [8].

4 Schluss mit dem digitalen Wecker: Warum Stille Gold ist

Hand aufs Herz: Nichts ruiniert die Moral eines Teams schneller als ein Telefon, das nachts um 3 Uhr vibriert, nur um zu melden: „CPU-Auslastung bei 85 %“ – und fünf Minuten später von selbst wieder auf „OK“ springt.

Dieses Phänomen hat einen Namen: **Alarmmüdigkeit** (Alert Fatigue) [9].

Wenn alles ein Notfall ist, ist nichts mehr ein Notfall. In der frühen Phase von *frag.jetzt* neigten wir dazu, auf alles zu lauschen. Ein Service antwortet nicht sofort? Alarm! Die Datenbank schreibt viele Logs? Alarm! Das Ergebnis: Wir ignorierten die Benachrichtigungen irgendwann, genau wie man das Rauschen einer Klimaanlage ausblendet. Und genau dann passierte der echte Ausfall.

Wir haben unser Alarm-Konzept radikal umgestellt – weg von „Rauschen“, hin zu „Handlung“.

- **Regel 1: Weck mich nur, wenn der Nutzer leidet.** Ein hoher CPU-Wert ist kein Grund zur Panik, solange die Antworten schnell kommen. Wir haben gelernt: Alarmiere auf *Symptome*, nicht auf Ursachen. *Schlecht*: „Server Node 4 hat hohe Last.“ (Wen interessiert’s?) *Gut*: „Die 99 %-Latenz für das Absenden von Fragen ist auf über 2 Sekunden gestiegen.“ (Handlungsbedarf!)
- **Regel 2: Die „Luft anhalten“-Strategie.** Netzwerke haben Schluckauf. Das ist normal. Statt bei jedem verlorenen Paket sofort „Feuer!“ zu schreien, nutzen wir zeitliche Glättung. Unser System wartet jetzt: Besteht das Problem länger als zwei Minuten? Erst dann geht die Nachricht raus. Das eliminiert nervöse Fehlalarme („Flapping“) fast vollständig.
- **Regel 3: Kein Alarm ohne Spielbuch.** Ein Alarm, der nur sagt „Es ist kaputt“, ist wertlos. Er erzeugt Stress, aber keine Lösung. Wir haben

begonnen, **Runbooks** direkt mit den Alerts zu verknüpfen. Wenn der „Datenbank-Pool erschöpft“-Alarm kommt, liefert er den Link zur Anleitung gleich mit: „1. Prüfe offene Verbindungen, 2. Starte Service X neu.“ Das spart in der Nacht die entscheidenden Minuten, in denen man sonst schlaftrunken nach der Ursache sucht [9].

Unser Fazit: Ein gutes Observability-System ist wie ein guter Rauchmelder: Es schweigt die meiste Zeit. Aber wenn es sich meldet, dann schnappt man sich den Feuerlöscher – und nicht die Ohropax.

5 Fazit: Wenn aus Stille Vertrauen wird

Kehren wir zurück in Hörsaal 1. Wieder zücken 500 Studierende ihre Smartphones. Wieder stellt der Dozent eine Frage über *frag.jetzt*. Und wieder gibt es im Hintergrund vielleicht ein technisches Problem – eine Datenbankabfrage dauert 200 Millisekunden zu lang.

Der Unterschied zu früher? Niemand merkt es.

Bevor der Ladebalken auf der Leinwand überhaupt ins Stocken gerät, hat unser Observability-Stack das Symptom („Latenz steigt im Backend an“) bereits erfasst. Prometheus hat den Trend erkannt, noch bevor der Grenzwert gerissen wurde. Ein Blick in das Tracing-Dashboard hat dem Team gezeigt: „Aha, der Connection-Pool zur Datenbank läuft voll.“ Das Runbook lieferte die Lösung, noch bevor der erste Nutzer frustriert die App schließen konnte.

Das ist der Kern von Observability. Es geht nicht um bunte Dashboards an der Wand, die beeindruckend aussehen. Es geht darum, das „Unbekannte Unbekannte“ beherrschbar zu machen. Wir haben gelernt, dass wir Fehler in verteilten Systemen nicht verhindern können – Systeme *werden* ausfallen. Aber wir können entscheiden, wie lange wir brauchen, um sie zu verstehen.

Der nächste Schritt: Von „Best Effort“ zu Service Level Objectives (SLOs)

Sind wir fertig? Nein. Observability ist kein Projekt, sondern ein Prozess. Aktuell messen wir, was *ist*. Der nächste Schritt, den wir bereits ins Auge fassen, ist die Definition von dem, was *sein darf*. Mit **Service Level Objectives (SLOs)** wollen wir festlegen: „99 % aller Fragen müssen in unter 500 ms beantwortet sein“. Das zwingt uns, Technik nicht mehr technisch, sondern vom Nutzer her zu denken. Wenn wir dieses Ziel verfehlen, stoppen wir neue Features und fixen die Stabilität [3].

Am Ende ist *frag.jetzt* mehr als Code. Es ist die Interaktion zwischen Menschen. Und unsere Aufgabe ist

es, sicherzustellen, dass die Technik dieser Interaktion niemals im Weg steht.

wurden KI-Tools (ChatGPT 5.2, Google Gemini Pro 3, Consensus, Elicit) zur Unterstützung bei Recherche und Formulierung eingesetzt. Die inhaltliche Verantwortung liegt beim Autor.

Quellen und weiterführende Literatur

- [1] TH Mittelhessen und arsnova. *frag.jetzt – Open-Source Q&A-Plattform*. 2025. URL: <https://gitlab.arsnova.eu/arsnova/frag.jetzt/-/blob/staging/README.md> (besucht am 20.01.2026).
- [2] S. Fischer u. a. „An Overview of Microservice-Based Systems Used for Evaluation in Testing and Monitoring: A Systematic Mapping Study“. In: (2024). DOI: 10.1145/3644032.3644445.
- [3] Rob Ewaschuk. *Monitoring Distributed Systems*. <https://sre.google/sre-book/monitoring-distributed-systems/>. Kapitel 6 in *Site Reliability Engineering*, herausgegeben von Betssy Beyer, Google. 2017.
- [4] Cindy Sridharan. *Distributed Systems Observability*. O'Reilly Media, 2018. URL: <https://unlimited.humio.com/rs/756-LMY-106/images/Distributed-Systems-Observability-eBook.pdf>.
- [5] Luca Giamattei u. a. „Monitoring tools for DevOps and microservices: A systematic grey literature review“. In: *Journal of Systems and Software* 208 (2024), S. 111906. DOI: 10.1016/j.jss.2023.111906.
- [6] A. Banerjee und R. Singh. „A Comparative Analysis of System Monitoring Architecture: Evaluating Prometheus, ELK Stack, and Custom Dashboards for Performance and Scalability“. In: *International Journal of Computer Applications* (2024). DOI: 10.21275/SR25917095133.
- [7] Olumese Anthony Abieba u. a. „Advanced Observability and Monitoring Practices for Enhancing Production Environment Reliability“. In: (2025). URL: <https://worldscientificnews.com/wp-content/uploads/2025/06/WSN-204-2025-270-296.pdf>.
- [8] Carlos Albuquerque und Filipe F Correia. „Tracing and Metrics Design Patterns for Monitoring Cloud-native Applications“. In: *arXiv preprint arXiv:2510.02991* (2025). URL: <https://arxiv.org/abs/2510.02991>.
- [9] Fatemeh Jalalvand u. a. „Alert prioritisation in security operations centres: A systematic survey on criteria and methods“. In: *ACM Computing Surveys* 57.2 (2024), S. 1–36. DOI: 10.1145/3695462.